

AirQualityAnalyzer — procesare în timp real a datelor de calitate a aerului

Tehnici de procesare a datelor de mari dimensiuni

Repository public: <https://github.com/dumibxd26/AirQualityAnalyzer>

1. Despre proiect

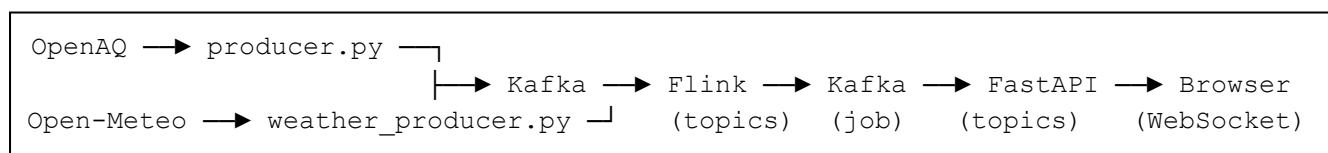
Proiectul construiește o conductă de procesare în flux pentru date de calitate a aerului. Citește măsurători de la stații publice, le combină cu date meteo pentru aceeași locație, calculează agregate pe ferestre de timp, ridică alerte când concentrațiile depășesc pragurile recomandate de OMS și trimite rezultatele către un panou de vizualizare care se actualizează pe măsură ce sosesc datele.

Întregul lanț rulează în containere Docker și folosește trei tehnologii care acoperă rolurile uzuale dintr-o platformă de date în flux:

- **Apache Kafka** — magistrala de mesaje care decuplează sursele de date de motorul de calcul. Fiecare tip de mesaj are propriul subiect (topic).
- **Apache Flink (PyFlink)** — motorul de procesare în flux care face agregările pe ferestre, joncțiunea dintre poluare și meteo și escaladarea alertelor.
- **FastAPI + WebSocket + frontend** — un pod care preia rezultatele din Kafka și le împinge în browser, plus un panou scris în JavaScript cu hartă (Leaflet) și grafice (Chart.js).

Motivul pentru care aceste date se pretează la procesare în flux: măsurătorile sosesc continuu, în ordine aproximativă, din zeci de stații, iar valoarea lor informativă este maximă cât sunt proaspete. O alertă de depășire a pragului are sens să fie ridicată în secunde, nu la sfârșitul zilei după o procesare în lot.

2. Arhitectura



Fluxul de date, etapă cu etapă:

1. **producer.py** interoghează API-ul OpenAQ v3 pentru șase poluanți și publică fiecare măsurătoare în subiectul `raw-air-quality`.
2. **weather_producer.py** ascultă `raw-air-quality` ca să afle ce stații există (cu latitudine și longitudine), apoi interoghează Open-Meteo pentru vremea curentă la fiecare stație și publică în `weather-stream`.
3. **flink_processor.py** consumă ambele subiecte, calculează agregate pe ferestre de un minut, le îmbogățește cu meteo, detectează depășirile de prag și scrie în subiectele `pollution-alerts`, `enriched-readings` și `critical-alerts`.
4. **backend/main.py** consumă subiectele de ieșire și le retransmite prin WebSocket către panou; expune și `/health` și `/snapshot` pentru starea curentă.
5. **frontend/** afișează harta stațiilor, graficele pe poluanți și lista de alerte, actualizate în timp real.

2.1 Subiectele Kafka

Brokerul rulează cu un singur nod plus Zookeeper. Subiectele sunt create explicit, cu numere de partiții alese în funcție de volum și de gradul de paralelism dorit în Flink:

Subiect	Partiții	Conținut
raw-air-quality	12	măsurători brute de la OpenAQ
weather-stream	12	vremea curentă pe stație de la Open-Meteo
enriched-readings	6	agregate pe minut + meteo (pentru panou)
pollution-alerts	6	depășiri de prag pe fereastră
critical-alerts	3	episoade escaladate (depășiri repetate)

Brokerul are două ascultătoare: unul intern (kafka:29092) pentru serviciile din Docker și unul extern (localhost:9092) pentru procesele de pe gazdă. Subiectul intern `__transaction_state` este configurat cu factor de replicare 1 și ISR minim 1, condiție necesară ca scrierile tranzacționale ale Flink să funcționeze pe un singur broker.

2.2 Jobul Flink

Jobul rulează cu un JobManager și două TaskManager-e, paralelism implicit 4 și checkpoint la fiecare 30 de secunde în mod `EXACTLY_ONCE`. Folosește timpul evenimentului (event time) cu watermark de 60 de secunde toleranță la dezordine. Deoarece cele 12 partiții ale fiecărui subiect nu primesc date uniform, partițiile inactive sunt marcate ca atare după 10 secunde, ca watermark-ul global să poată avansa.

Toate ieșirile sunt produse de un singur StatementSet cu patru destinații (alerte de poluare, citiri îmbogățite, alerte critice și o destinație de tip print pentru log).

3. Cum sunt preluate datele

3.1 OpenAQ (calitatea aerului)

producer.py interoghează endpoint-ul OpenAQ v3 pentru șase parametri:

Id OpenAQ	Parametru
2	pm25
1	pm10
7	no2
10	o3
9	so2
8	co

Intervalul de interogare este de 15 secunde. Câteva decizii de proiectare merită menționate, pentru că ele determină calitatea datelor care intră în pipeline:

- **De-duplicare.** OpenAQ `/latest` întoarce aceeași valoare la fiecare ciclu până când stația publică una nouă. Producătorul reține ultima pereche (locație, parametru) văzută și nu republică aceeași măsurătoare decât după ce expiră un interval (90 de secunde), pentru a păstra ferestrele Flink alimentate fără a inunda subiectul.
- **Eliminarea citirilor vechi.** `/latest` poate întoarce valori vechi de ani de la stații scoase din uz, care altfel ar strica watermark-ul. Citirile mai vechi de șase ore sunt aruncate.

- **Rescrierea marcajului de timp.** Marcajul mesajului Kafka este pus la timpul de ingestie, ca ferestrele pe timp de eveniment să se închidă prompt; timpul original al evenimentului este păstrat separat în `event_time`.
- **Filtrarea valorilor santinelă.** OpenAQ folosește valori speciale pentru „lipsă de date” (de exemplu 9999, -999). Acestea trec de praguri și produc alerte false. Producătorul respinge santinelele exacte plus limite superioare implauzibile pe poluant.

3.2 Open-Meteo (vremea)

`weather_producer.py` nu interoghează la întâmplare: se abonează la `raw-air-quality`, învață ce stații există și unde se află, apoi interoghează Open-Meteo pentru vremea curentă (temperatură, viteza și direcția vântului, umiditate, precipitații) la fiecare stație, la fiecare 60 de secunde. Rezultatul se publică în `weather-stream`. Acest al doilea flux există ca să facă posibilă joncțiunea dintre poluare și meteo în Flink.

4. Cum sunt definite alertele

Toată logica de alertare se află în jobul Flink și are trei niveluri.

4.1 Agregarea pe fereastră

Pe sursa de poluare se face o fereastră *tumbling* de un minut, grupată după (locatie, parametru), care produce media valorilor și numărul de eșantioane din fereastră. Doar valorile nenule și nenegative intră în medie.

4.2 Depășirea pragului

Media fiecărei ferestre este comparată cu un prag specific poluantului, inspirat de recomandările OMS:

Parametru	Prag
pm25	15
pm10	45
no2	25
o3	60
so2	40
co	4

Când media depășește pragul, se emite o alertă în `pollution-alerts`, clasificată pe severitate:

- **MODERATE** — peste prag;
- **HIGH** — peste de două ori pragul;
- **CRITICAL** — peste de trei ori pragul.

4.3 Escaladarea critică

Depășirile izolate nu sunt neapărat relevante. Pentru a prinde episoadele susținute, alertele sunt grupate într-o fereastră *tumbling* de cinci minute pe (locatie, parametru); dacă într-o fereastră apar cel puțin trei depășiri consecutive, se emite un singur eveniment în `critical-alerts`, cu valoarea de vârf și numărul de depășiri. Fereastra non-suprapusă garantează că un episod produce un singur eveniment critic, nu copii multiple.

4.4 Îmbogățirea cu meteo

În paralel, agregatele pe minut sunt unite cu fluxul meteo printr-o joncțiune pe interval (interval join): fiecare fereastră primește eșantionul meteo al aceleiași stații aflat în intervalul de ± 5 minute. Rezultatul ajunge în enriched-readings și alimentează panoul și analiza de corelație de mai jos.

5. Rezultate

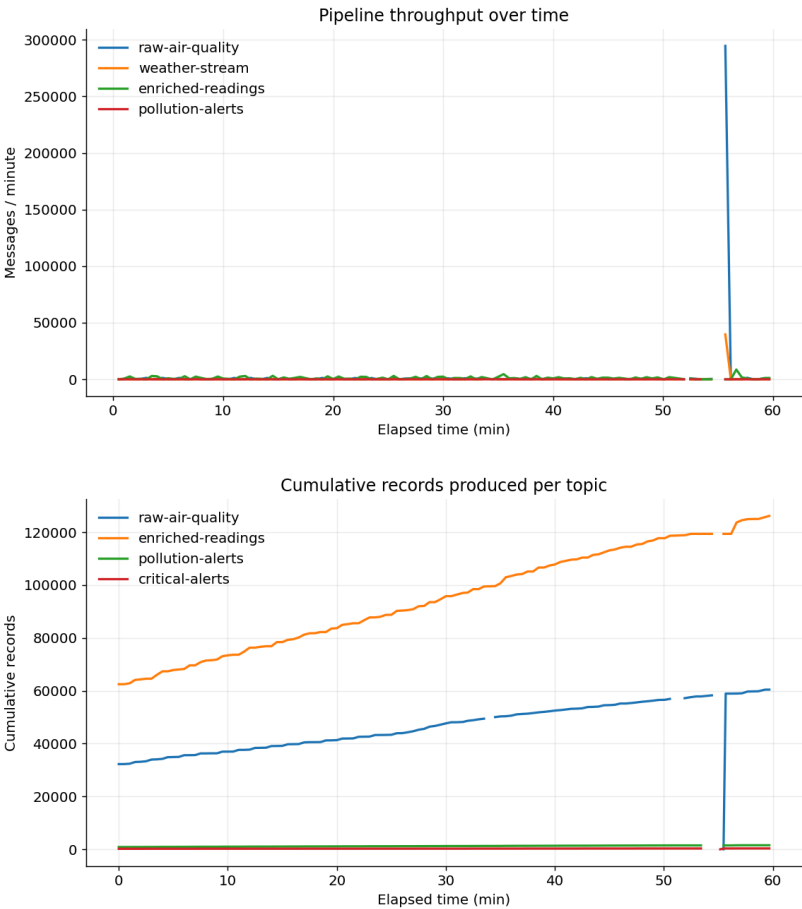
Pentru raport, pipeline-ul a rulat continuu și am colectat datele timp de o oră (31 mai 2026, 13:19–14:19 UTC) direct din subiectele Kafka. Volumul procesat în acest interval:

Subiect	Înregistrări noi în oră
raw-air-quality	~28.200
enriched-readings	~63.700
pollution-alerts	682
critical-alerts	140

Din fluxul de ieșire am capturat 62.908 ferestre îmbogățite, 488 de alerte de poluare și 101 evenimente critice. Distribuția pe severitate a alertelor a fost de 451 MODERATE și 37 HIGH; nicio fereastră nu a atins pragul CRITICAL (peste de trei ori limita), ceea ce e consistent cu o atmosferă fără episoade extreme în acel interval.

5.1 Debit și volum cumulat

Numărul de mesaje a crescut constant pe toate subiectele pe durata orei, fără opriri, ceea ce confirmă că pipeline-ul s-a menținut stabil.



5.2 Statistici pe poluant

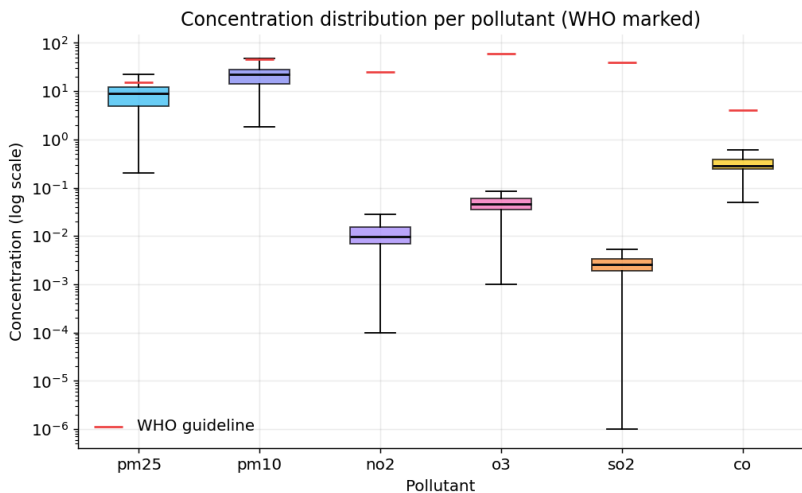
Tabelul de mai jos rezumă cele 62.908 de ferestre îmbogățite, pe poluant. Pentru pm25 și pm10 valorile sunt în $\mu\text{g}/\text{m}^3$ și se compară direct cu pragul OMS. Pentru no2, o3, so2 și co, sursa OpenAQ raportează în ppm, deci comparația absolută cu pragul nu se aplică direct; fiecare poluant este analizat separat.

Poluant	Ferestre	Medie	Mediană	Max	P95	% peste prag
pm25	5.240	8.99	8.98	36.60	18.00	13.0%
pm10	11.930	23.76	22.00	85.50	50.00	6.9%
no2	17.831	0.012	0.010	0.047	0.028	0%
o3	9.218	0.046	0.046	0.085	0.070	0%
so2	9.347	0.003	0.002	0.008	0.005	0%
co	9.342	0.337	0.280	2.200	0.650	0%

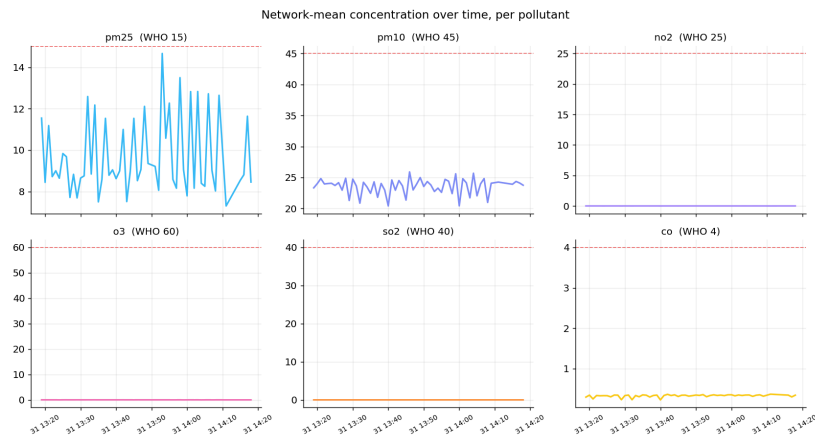
Observația principală: dintre poluanții măsurați în $\mu\text{g}/\text{m}^3$, pm25 este cel care depășește pragul cel mai des — 13% dintre ferestre, cu un vârf de 36,6 $\mu\text{g}/\text{m}^3$, adică de 2,4 ori limita OMS. pm10 depășește în 6,9% dintre ferestre, cu vârf de 85,5 $\mu\text{g}/\text{m}^3$.

5.3 Distribuția concentrațiilor

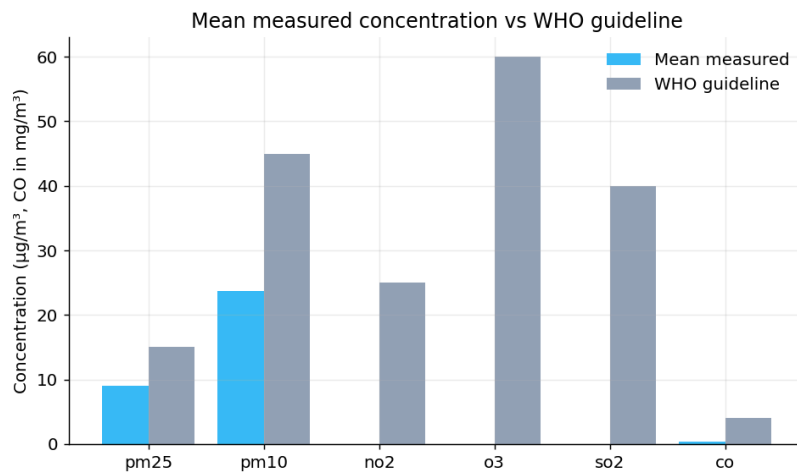
Graficul de tip boxplot arată distribuția pe poluant (scară logaritmică, cu pragul marcat). Mediile și mediile relativ apropiate indică distribuții fără asimetrii mari, dar cu valori extreme la pm25 și pm10.



5.4 Evoluția în timp pe poluant

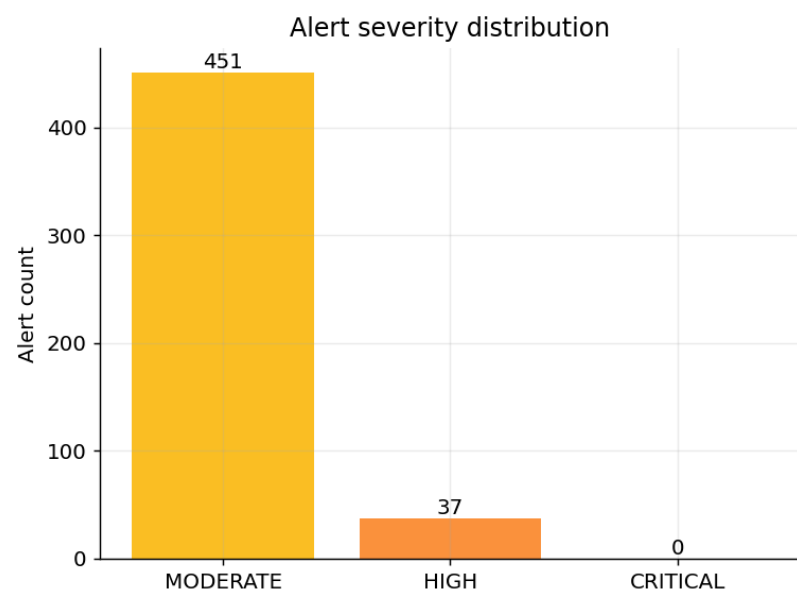
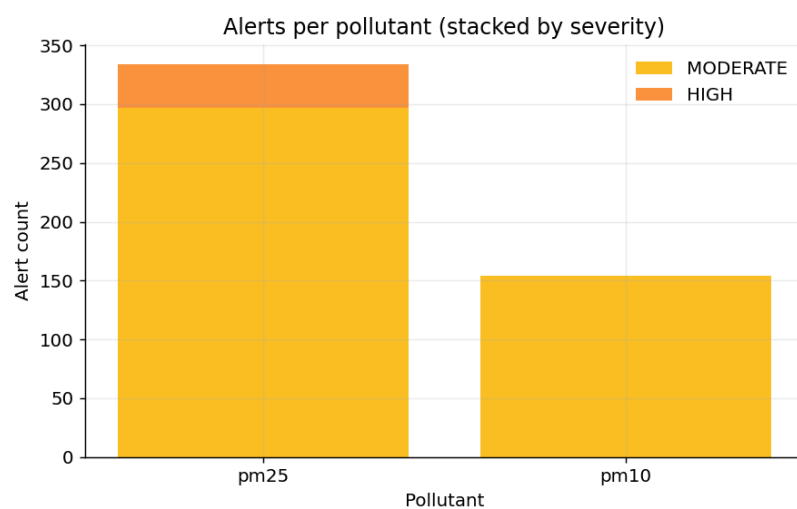


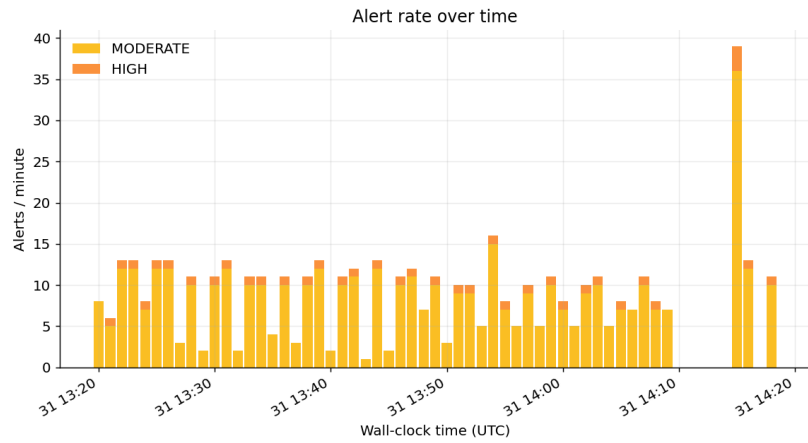
5.5 Concentrație medie față de pragul OMS



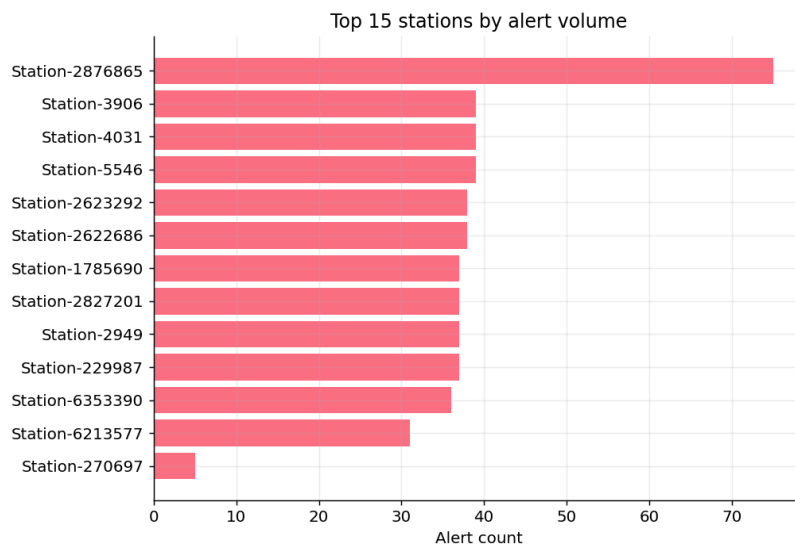
5.6 Alerte

Repartiția alertelor pe poluant și pe severitate, plus evoluția lor în timp:





Stațiile cu cele mai multe alerte:

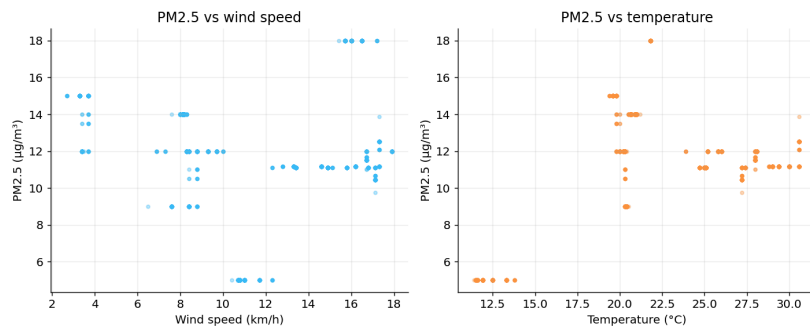


5.7 Corelația cu vremea

Folosind ferestrele îmbogățite, am calculat corelația Pearson dintre concentrația fiecărui poluant și trei variabile meteo:

Poluant	Vânt	Temperatură	Umiditate	n
pm25	0.095	0.370	-0.215	2.037
pm10	0.131	0.525	0.502	10.731
no2	-0.118	0.026	0.244	15.538
o3	-0.263	0.022	0.020	4.398
so2	-0.558	-0.384	0.100	5.295
co	-0.502	-0.413	-0.156	4.497

Câteva relații se desprind din date: so2 și co scad când vântul crește (corelație -0,56, respectiv -0,50), ceea ce e consistent cu dispersia poluanților de către vânt. pm10 crește cu temperatura și umiditatea (0,53 și 0,50).



6. Garanții de procesare

Pipeline-ul este configurat pentru livrare exactly-once de la capăt la capăt: checkpointing activat la 30 de secunde, plus destinații Kafka tranzacționale, cu prefix de id de tranzacție distinct pe fiecare destinație, ca subtask-urile paralele să nu intre în coliziune. Jobul a rulat pe durata orei cu 28 de task-uri, fără reporniri.

7. Rularea proiectului

Întregul stack pornește cu Docker Compose:

```
docker compose up -d
```

Aceasta ridică Zookeeper, brokerul Kafka, jobul de inițializare a subiectelor, clusterul Flink (JobManager + 2 TaskManager), cei doi producători, backend-ul FastAPI și frontend-ul. Panoul este disponibil la <http://localhost:8080>, iar interfața web Flink la <http://localhost:8081>.

Scriptul de colectare a statisticilor și cel de generare a graficelor se află în directorul `report/` și se conectează la ascultătorul extern al Kafka (`localhost:9092`).

8. Structura codului

Fișier	Rol
<code>producer.py</code>	OpenAQ → raw-air-quality
<code>weather_producer.py</code>	Open-Meteo → weather-stream
<code>flink_processor.py</code>	agregare, joncțiune, alerte
<code>backend/main.py</code>	pod Kafka → WebSocket + REST
<code>frontend/</code>	panou (hartă, grafice, alerte)
<code>docker-compose.yml</code>	orchestrare
<code>report/collect_stats.py</code>	colectare statistici din Kafka
<code>report/make_plots.py</code>	generarea figurilor

Codul sursă complet este disponibil în repository-ul public: <https://github.com/dumibxd26/AirQualityAnalyzer>